Threadwatch: Parallel Anomaly Detection in Long Server-Metric Time Series with OpenMP, MPI and Hybrid Parallelism

A Controlled Evaluation of Speedup, Scalability and Detection Quality

Bourahla Ismail Ahmed Abdelaziz

Module: Acceleration of Artificial Intelligence Algorithms by Parallel and Distributed Computing

Project sheet (week 1 deliverable) · Project 4: time series

November 3, 2026

Abstract. Monitoring systems must detect anomalies in server metrics as fast as the data arrives. This project studies how parallel computing reduces detection time and whether it preserves result quality. Four detectors (rolling z-score, rolling median/MAD, K-Means, Isolation Forest) will be run in five modes: sequential Python, sequential C, OpenMP, MPI and hybrid MPI+OpenMP, on real and synthetic series of increasing size. Measurements cover run time, speedup, efficiency, throughput and F1. This sheet fixes the research question, hypotheses, protocol and schedule; no result is available yet.

Keywords: anomaly detection; time series; OpenMP; MPI; hybrid; performance.

1. Introduction and problem

Monitoring tools (AIOps) continuously watch metrics such as CPU load, latency or request rate. Detecting anomalies in very long series needs a lot of computation, and detection must stay fast enough to keep up with the data stream. The project does not propose a new algorithm: it measures how parallelism speeds up existing detectors and under which conditions the gain is real.

Research question. For which series sizes and thread or process counts does parallel detection pay off, and does detection quality stay identical?

2. Objectives

- Build a sequential Python reference and measure its run time.
- Rewrite the compute kernels in a C SDK (DLL) called from Python and measure the single-thread time, T_1.
- Parallelise with OpenMP, MPI and then a hybrid approach.
- Measure time, speedup, efficiency, throughput and quality for several sizes and configurations.
- Separate the language effect (Python to C) from the parallel effect, so that speedup is not inflated.
- Present a demonstration with a live web dashboard.

3. Hypotheses

The hypotheses are falsifiable; results will be reported either way.

H1. Kernels with $O(w)$ work per window (z-score, MAD) keep a parallel efficiency above 70% up to the physical core count.

H2. Parallel outputs are identical to single-thread outputs, so F1 does not depend on the number of threads or ranks.

H3. Below a certain series size, overhead makes parallel runs slower than one thread; the crossover size will be measured.

H4. On a single machine MPI does not beat OpenMP (message cost); hybrid helps only if q processes $\times$ p threads is well tuned.

4. Data and method

Data. Real series from NAB (AWS CloudWatch metrics) and SMD (28 machines, 38 metrics), plus a labelled synthetic generator (trend, cycles, AR(1) noise, spikes and level shifts) for scaling tests. Sizes, licences and citations will be verified when downloading.

Execution modes. All five modes call the same kernels, so any difference comes from how the work is run. A series of n points is cut into $n_w = \lfloor (n-w)/s \rfloor + 1$ windows of length w and stride s. In MPI, each process receives its windows with an overlap of $w - s$ points, so that the gathered result equals the sequential one.

Measurement protocol. Lab 1 showed, on our Windows machine, that `clock()` advances in 1 ms steps and that the same loop ($N = 10^7$) varied from 17 to 29 ms between runs. We will therefore use a high-resolution timer, a warm-up run, at least 5 runs per configuration (median and spread), an idle machine and a folder outside OneDrive synchronisation.

Tools. Python, NumPy, pandas, scikit-learn; C11 with GCC (MSYS2 UCRT64) and OpenMP; MS-MPI with mpi4py; FastAPI and HTML/JavaScript for the dashboard; Git.

5. Success metrics

Metric	Definition and target
Run time	T_p, median of at least 5 runs
Speedup	$S(p) = T_1/T_p$, for every size
Efficiency	$E(p) = S(p)/p$ and Karp-Flatt serial fraction
Throughput	samples processed per second
Quality	precision, recall, F1, event recall; unchanged by parallelism
Correctness	C output vs NumPy reference: relative error $< 10^{-5}$
Determinism	parallel output identical to single-thread output
Language effect	$G = T_{\text{Python}}/T_1$, reported separately

6. Scope

In scope: univariate series; a single Windows machine; public and synthetic datasets; batch detection and live replay. **Out of scope:** GPU, multi-machine clusters, Spark, deep-learning detectors, energy measurement, production-use claims.

7. Deliverables and schedule

8. Risks

DLL loading on Windows (32/64-bit, dependencies), MPI installation, and noisy timings on a laptop (mitigated by the protocol above).

Wk	Expected deliverable and implementation
1	Project sheet: D-01, Git repository, “hello DLL” and “hello MPI” tests
2	State of the art and data: D-02, 4 to 6 sources read, datasets, generator
3	First working version: D-03, Python detectors, F1, first timings
4	Sequential results: D-04, single-thread C SDK, T_{Python} and T_1
5	Parallel program: D-05 and D-06, OpenMP, then MPI and hybrid
6	Results tables: D-08, benchmark campaign, `results.csv`
7	Graphs and interpretation: D-09, figures, reading of H1 to H4
8	Final report and demonstration: D-10, report, dashboard (D-07), release 1.0